

Practical – 1

Aim:

Implementation of preprocessing of text with NLTK (Tokenization, Stemming, Lemmatization, and Removal of Stop Words in NLP).

Theory:

Natural Language Processing (NLP) requires raw text to be cleaned and standardized before further analysis. The main pre-processing techniques are:

1. Tokenization

- Breaks text into smaller units called tokens (words, subwords, or phrases).
- Example:
"NLP is fun" → ["NLP", "is", "fun"]

2. Stemming

- Reduces words to their base/root form by chopping off suffixes.
- Not always dictionary valid words.
- Example:
 - "starting" → "start"
 - "running" → "run"
 - "happily" → "happili"

3. Lemmatization

- More accurate than stemming. Uses linguistic rules and vocabulary.
- Produces valid dictionary words.
- Example:
 - "went" → "go"
 - "better" → "good"

4. Stop Word Removal

- Removes common, less meaningful words to reduce noise.
- Examples: "the", "is", "and", "with", "for"

Code:

```
import nltk  
  
from nltk.tokenize import word_tokenize  
  
from nltk.corpus import stopwords  
  
from nltk.stem import PorterStemmer, WordNetLemmatizer
```

```
# Download required NLTK data files

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')

# Sample Text

text = "Natural Language Processing (NLP) is a fascinating field of Artificial Intelligence. It involves tokenizing, stemming, and lemmatizing words."

print("Original Text:\n", text)

# 1. Tokenization

tokens = word_tokenize(text)

print("\n1. Tokenized Words:\n", tokens)

# 2. Removing Stop Words

stop_words = set(stopwords.words("english"))

filtered_tokens = [word for word in tokens if word.lower() not in stop_words and word.isalpha()]

print("\n2. Tokens after Stop Words Removal:\n", filtered_tokens)

# 3. Stemming

stemmer = PorterStemmer()

stemmed_words = [stemmer.stem(word) for word in filtered_tokens]

print("\n3. Stemmed Words:\n", stemmed_words)

# 4. Lemmatization

lemmatizer = WordNetLemmatizer()

lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_tokens]

print("\n4. Lemmatized Words:\n", lemmatized_words)
```

Output:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
Original Text:
Natural Language Processing (NLP) is a fascinating field of Artificial Intelligence. It involves tokenizing, stemming, and lemmatizing words.

1. Tokenized Words:
['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'a', 'fascinating', 'field', 'of', 'Artificial', 'Intelligence', '.', 'It', 'involves', 'tokenizing',

2. Tokens after Stop Words Removal:
['Natural', 'Language', 'Processing', 'NLP', 'fascinating', 'field', 'Artificial', 'Intelligence', 'involves', 'tokenizing', 'stemming', 'lemmatizing', 'words']

3. Stemmed Words:
['natur', 'languag', 'process', 'nlp', 'fascin', 'field', 'artifici', 'intellig', 'involv', 'token', 'stem', 'lemmat', 'word']

4. Lemmatized Words:
['Natural', 'Language', 'Processing', 'NLP', 'fascinating', 'field', 'Artificial', 'Intelligence', 'involves', 'tokenizing', 'stemming', 'lemmatizing', 'word']
```

Practical – 2

Aim:

Implementation to convert text into word count vectors using Scikit-Learn's CountVectorizer.

Theory:

In Natural Language Processing (NLP), raw text cannot be fed directly into machine learning models. It must be converted into numerical form.

One common method is Bag-of-Words (BoW) using CountVectorizer.

1. CountVectorizer

- Converts a collection of text documents into a matrix of token counts.
- Each row represents a document.
- Each column represents a word (feature) from the vocabulary.
- Each cell contains the frequency of that word in the document.

Example:

Text corpus = ["NLP is fun", "NLP is powerful"]

Vocabulary = ["fun", "is", "nlp", "powerful"]

Word Count Vectors = [[1, 1, 1, 0], # "NLP is fun"

[0, 1, 1, 1]] # "NLP is powerful"

Code:

```
from sklearn.feature_extraction.text import CountVectorizer
# Sample Data
data = [
    "Hello this is Narendra Modi and this code for count vectorizer.",
    "and here is implementation of text to word count vectors",
]
# Initialize CountVectorizer
vectorizer = CountVectorizer()
# Fit the model and transform the data
vectorizer.fit(data)
word_count_vectors = vectorizer.transform(data)
# Convert sparse matrix to array
word_count_vectors_array = word_count_vectors.toarray()
# Get the vocabulary
vocabulary = vectorizer.get_feature_names_out()
```

```
# Display Word Count Vectors
print("Word Count Vectors:")
print(word_count_vectors_array)

# Display Vocabulary with counts for the first document
print("\nVocabulary and Counts (Document 1):")
for word, count in zip(vocabulary, word_count_vectors_array[0]):
    print(f"{word}: {count}")

# Example: Transform a new piece of text
new_data = ["active document"]
new_vector = vectorizer.transform(new_data).toarray()
print("\nWord Count Vector for New Data:")
print(new_vector)
```

Output:

```
Word Count Vectors:
[[1 1 1 1 1 0 0 1 1 1 0 0 2 0 1 0 0]
 [1 0 1 0 0 1 1 1 0 0 1 1 0 1 0 1 1]]

Vocabulary and Counts (Document 1):
and: 1
code: 1
count: 1
for: 1
hello: 1
here: 0
implementation: 0
is: 1
modi: 1
narendra: 1
of: 0
text: 0
this: 2
to: 0
vectorizer: 1
vectors: 0
word: 0

Word Count Vector for New Data:
[[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]]
```

Practical – 3

Aim:

Implementation to convert text into word frequency vectors using Scikit-Learn's TfidfVectorizer.

Theory:

While CountVectorizer only counts how many times a word appears in a document, TF-IDF (Term Frequency – Inverse Document Frequency) goes one step further. It assigns weights to words based on:

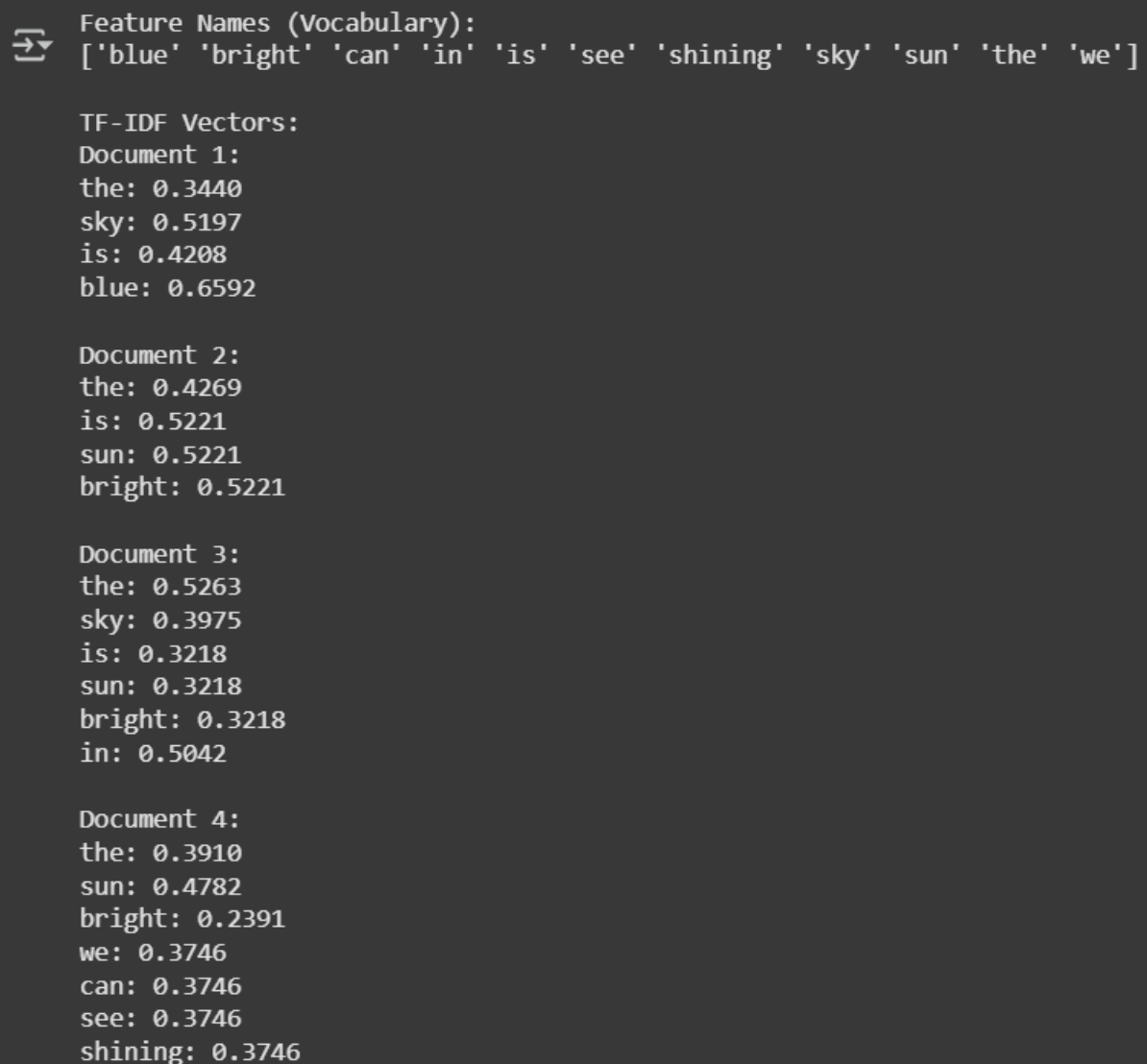
1. Term Frequency (TF):
 - How often a word appears in a document.
 - Higher frequency → higher weight (but normalized).
2. Inverse Document Frequency (IDF):
 - Reduces the weight of common words that appear in many documents.
 - Gives importance to rare but meaningful words.

Code:

```
from sklearn.feature_extraction.text import TfidfVectorizer
# Sample text data
documents = [
    "The sky is blue",
    "The sun is bright",
    "The sun in the sky is bright",
    "We can see the shining sun, the bright sun"
]
# Create TfidfVectorizer instance
vectorizer = TfidfVectorizer()
# Fit the vectorizer to the documents and transform the data
X = vectorizer.fit_transform(documents)
# Get the feature names (unique words)
feature_names = vectorizer.get_feature_names_out()
# Print the feature names
print("Feature Names (Vocabulary):")
print(feature_names)
print()
# Print the TF-IDF vectors for each document
```

```
print("TF-IDF Vectors:")
for i in range(len(documents)):
    print(f"Document {i+1}:")
    for feature_index in X[i].indices:
        print(f"{feature_names[feature_index]}: {X[i, feature_index]:.4f}")
    print()
```

Output:

A terminal window with a dark background and light gray text. It shows the output of a TF-IDF vectorization process. The first line is a prompt character followed by 'Feature Names (Vocabulary):' and a list of words in single quotes. The subsequent lines show 'TF-IDF Vectors:' followed by four documents, each with its word features and values formatted to four decimal places.

```
⇒ Feature Names (Vocabulary):
['blue' 'bright' 'can' 'in' 'is' 'see' 'shining' 'sky' 'sun' 'the' 'we']

TF-IDF Vectors:
Document 1:
the: 0.3440
sky: 0.5197
is: 0.4208
blue: 0.6592

Document 2:
the: 0.4269
is: 0.5221
sun: 0.5221
bright: 0.5221

Document 3:
the: 0.5263
sky: 0.3975
is: 0.3218
sun: 0.3218
bright: 0.3218
in: 0.5042

Document 4:
the: 0.3910
sun: 0.4782
bright: 0.2391
we: 0.3746
can: 0.3746
see: 0.3746
shining: 0.3746
```

Practical – 4

Aim:

Implementation to convert text into fixed-length integer-based feature vectors using Scikit-Learn's HashingVectorizer.

Theory:

Unlike CountVectorizer or TfidfVectorizer, which build and store a vocabulary, the HashingVectorizer uses a hash function to directly map words to indices in a fixed-length vector.

1. Hashing Mechanism:

- Each word is hashed into an integer index.
- Index corresponds to a position in the vector.

2. Advantages:

- Memory efficient (no vocabulary dictionary).
- Works well on large-scale and streaming data.

3. Limitations:

- Cannot retrieve the original words (no reverse mapping).
- Possible hash collisions (two words mapping to the same index).

Code:

```
# Step 1: Import the required library
from sklearn.feature_extraction.text import HashingVectorizer

# Step 2: Define the input text data (a list of documents)
documents = [
    "Machine learning is fascinating",
    "Deep learning is a part of machine learning.",
    "Natural language processing involves machine learning."]

# Step 3: Create the HashingVectorizer object
vectorizer = HashingVectorizer(n_features=10, alternate_sign=False)

# Step 4: Transform the documents into fixed-length hashed feature vectors
hashed_vectors = vectorizer.transform(documents)

# Step 5: Convert the sparse matrix to a dense array and print the result
hashed_vectors_array = hashed_vectors.toarray()

print("Hashed Vectors (Each document mapped to 10 unique integer-based features):\n")

for i, vector in enumerate(hashed_vectors_array):
```

```
print(f"Document {i+1}: {vector}\n")
```

Output:

```
➡ Hashed Vectors (Each document mapped to 10 unique integer-based features):  
  
Document 1: [0.  0.5 0.5 0.  0.5 0.  0.  0.5 0.  0. ]  
  
Document 2: [0.          0.          0.60302269 0.          0.60302269 0.  
0.30151134 0.30151134 0.30151134 0.          ]  
  
Document 3: [0.35355339 0.          0.35355339 0.          0.70710678 0.35355339  
0.          0.35355339 0.          0.          ]
```


Practical – 5

Aim:

Use the Keras deep learning library to split words with `text_to_word_sequence`.

Theory:

Before feeding text into deep learning models, we need to tokenize and normalize it. Keras provides a convenient utility:

- `text_to_word_sequence()`
 - Converts text into a sequence of lowercase words.
 - Removes punctuation by default.
 - Produces a clean list of tokens.

Code:

```
from nltk.corpus import stopwords
from tensorflow.keras.preprocessing.text import text_to_word_sequence

# Our text document
text = "This text is used here just for demonstration purposes only and I am student from Parul University Vadodara."

# Step 1: Tokenize the text using Keras
words = text_to_word_sequence(text)

# Step 2 (Optional): Remove stopwords using NLTK
stop_words = set(stopwords.words('english'))
filtered_words = [word for word in words if word not in stop_words]

# Step 3: Calculate vocabulary size
vocabulary_size = len(set(filtered_words))

print("Tokenized Words:")
print(words)
print("\nWords After Stop Word Removal:")
print(filtered_words)
print("\nVocabulary Size (after stopword removal):", vocabulary_size)
```

Output:

```
Tokenized Words:
['this', 'text', 'is', 'used', 'here', 'just', 'for', 'demonstration', 'purposes', 'only', 'and', 'i', 'am', 'student', 'from', 'parul', 'university', 'vadodara']

Words After Stop Word Removal:
['text', 'used', 'demonstration', 'purposes', 'student', 'parul', 'university', 'vadodara']

Vocabulary Size (after stopword removal): 8
```

Practical – 6

Aim:

Use the Keras deep learning library to encode text using the `one_hot` method.

Theory:

In deep learning for NLP, words must be represented numerically before they can be used by models.

- One-hot encoding (in Keras):
 - The `one_hot()` function hashes each word to an integer value within a fixed vocabulary size.
 - It does not create a full binary vector (as in classical one-hot encoding), but instead assigns an integer index to each word.
 - Vocabulary size is usually predefined (e.g., 50, 100, 5000).

Steps:

1. Tokenize text into words.
2. Define vocabulary size.
3. Encode words into integer indices using `one_hot()`.

Code:

```
from nltk.corpus import stopwords

from tensorflow.keras.preprocessing.text import one_hot, text_to_word_sequence

# Step 1: Define your text
text = "This text is used here just for demonstration purposes only, I am STUDENT from Parul University Vadodara."

# Step 2: Tokenize the text (convert to lower-case & split into words)
words = text_to_word_sequence(text)

# Step 3: Create vocabulary size (optional: use len(set(words)) or a predefined size)
vocabulary_size = len(set(words)) # dynamic based on unique words

# Step 4: One-hot encode the text using Keras
one_hot_encoded = one_hot(text, vocabulary_size)

print("Tokenized Words:")
print(words)
print("\nOne-Hot Encoded Words (as integers):")
print(one_hot_encoded)
print("\nVocabulary Size:", vocabulary_size)
```

Output:



Tokenized Words:

```
['this', 'text', 'is', 'used', 'here', 'just', 'for', 'demonstration', 'purposes', 'only', 'i', 'am', 'student', 'from', 'parul', 'university', 'vadodara']
```

One-Hot Encoded Words (as integers):

```
[14, 7, 8, 4, 6, 10, 4, 3, 13, 8, 12, 6, 12, 7, 9, 15, 9]
```

Vocabulary Size: 17

Practical – 7

Aim:

Use the Keras deep learning library to perform hash encoding with `hashing_trick`.

Theory:

When dealing with large text datasets, storing a full vocabulary can be memory-intensive.

- `hashing_trick()`:
 - Maps words directly to integers using a hashing function (md5, hash, etc.).
 - Avoids storing vocabulary in memory.
 - Requires defining a hash space size (number of buckets).

Code:

```
# Step 1: Import necessary functions from Keras
from tensorflow.keras.preprocessing.text import hashing_trick,
text_to_word_sequence

# Step 2: Define your input text
text = "The quick brown fox jumped over the lazy dog dog."

# Step 3: Tokenize the text into words to find the vocabulary size
words = set(text_to_word_sequence(text))
vocab_size = len(words)
print("Output:")
print("Unique words:", words)
print("Vocabulary size:", vocab_size)

# Step 4: Use `hashing_trick` to encode text as integers
# We multiply vocab_size by 1.3 to reduce word collisions
hash_space = round(vocab_size * 1.3)
encoded = hashing_trick(text, hash_space, hash_function="md5")
print("Original Text:", text)
print("Hashed Encoding:", encoded)
```

Output:

```
➡ Output:
Unique words: {'jumped', 'brown', 'lazy', 'over', 'the', 'fox', 'dog', 'quick'}
Vocabulary size: 8
Original Text: The quick brown fox jumped over the lazy dog dog.
Hashed Encoding: [6, 4, 1, 2, 7, 5, 6, 2, 6, 6]
```

Practical – 8

Aim:

Use the Keras deep learning library to demonstrate the Tokenizer API.

Theory:

The Keras Tokenizer is a powerful utility for text preprocessing:

1. `fit_on_texts(texts)`
 - Builds the word index (a dictionary mapping each word to a unique integer).
2. `texts_to_sequences(texts)`
 - Converts each sentence into a sequence of integers, based on the word index.
3. `word_index`
 - Provides the mapping of words to their assigned integer IDs.

Code:

```
from nltk.corpus import stopwords
import keras
from tensorflow.keras.preprocessing.text import Tokenizer
tokenizer = Tokenizer()
texts = [
    "This is random text I am STUDENT from Parul University Vadodara.",
    "And here comes the second one.",
    "Finally, this is the third sentence."
]
tokenizer.fit_on_texts(texts)
# Convert a list of texts to a sequence of integers
sequences = tokenizer.texts_to_sequences(texts)
# Print the sequences
print("Output:")
print(sequences)
```

Output:

```
Output:
[[1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12], [13, 14, 15, 3, 16, 17], [18, 1, 2, 3, 19, 20]]
```

Practical – 9

Aim:

Perform Sentiment Analysis on a dataset (IMDb movie reviews) using LSTM with Keras.

Theory:

Sentiment Analysis is the process of determining whether a piece of text expresses a positive or negative opinion.

- We use the IMDb dataset:
 - 25,000 training reviews and 25,000 test reviews.
 - Each review is already labeled as positive (1) or negative (0).
- Steps in Workflow:
 1. Load dataset with integer-encoded words.
 2. Pad sequences to equal length for batch processing.
 3. Build LSTM model with embedding layer for word representation.
 4. Train the model on training data.
 5. Evaluate performance on test data.
- LSTM (Long Short-Term Memory):
 - A special kind of RNN that captures long-term dependencies in text.
 - Effective for sequential data like reviews, tweets, etc.

Code:

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
# Step 1: Load the IMDb Movie Review Dataset
max_features = 10000 # Number of words to consider as features (vocabulary size)
maxlen = 100 # Cut off texts after this number of words (max sequence length)
embedding_dim = 128 # Embedding vector size
# Load data and split into training and test sets
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)
```

```
# Step 2: Preprocess the Data (Pad sequences to have equal length)
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)

# Step 3: Build the LSTM Model
model = Sequential()
model.add(Embedding(max_features, embedding_dim, input_length=maxlen))
model.add(LSTM(128, dropout=0.2, recurrent_dropout=0.2)) # LSTM layer
model.add(Dense(1, activation="sigmoid")) # Output layer for binary classification
(positive/negative)

# Step 4: Compile the Model
model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])

# Step 5: Train the Model
batch_size = 32
epochs = 3

model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs,
validation_data=(x_test, y_test))

# Step 6: Evaluate the Model
score, accuracy = model.evaluate(x_test, y_test, batch_size=batch_size)
print(f"Test accuracy: {accuracy * 100:.2f}%")
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ————— 1s 0us/step
Epoch 1/3
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
warnings.warn(
782/782 ————— 266s 333ms/step - accuracy: 0.7007 - loss: 0.5516 - val_accuracy: 0.7791 - val_loss: 0.4718
Epoch 2/3
782/782 ————— 257s 327ms/step - accuracy: 0.8558 - loss: 0.3425 - val_accuracy: 0.8403 - val_loss: 0.3633
Epoch 3/3
782/782 ————— 261s 327ms/step - accuracy: 0.9013 - loss: 0.2484 - val_accuracy: 0.8352 - val_loss: 0.3803
782/782 ————— 35s 44ms/step - accuracy: 0.8322 - loss: 0.3849
Test accuracy: 83.52%
```

Practical – 10

Aim:

Perform an experiment using the Gensim Word2Vec library to generate word embeddings and explore similar words.

Theory:

- Word Embeddings are dense vector representations of words that capture semantic meaning. Words with similar meanings are placed closer in vector space.
- Word2Vec (Google, 2013) is a neural network-based model for learning embeddings. It has two main architectures:
 - CBOW (Continuous Bag of Words) → Predicts a word based on surrounding context.
 - Skip-Gram → Predicts the context given a word.
- Gensim is a Python library for efficient Word2Vec training and querying.

Code:

```
from gensim.models import Word2Vec
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import nltk
nltk.download('punkt')
data = [
    "Hello I am a student of from Parul University Vadodara.",
    "Word embeddings are useful for natural language processing.",
    "Gensim is a popular library for word embeddings." ]
tokenized_data = [word_tokenize(sentence.lower()) for sentence in data]
model = Word2Vec(sentences=tokenized_data, vector_size=100, window=5,
min_count=1, sg=0)
```



```
vector = model.wv[&#39;word&#39;]
similar_words = model.wv.most_similar(&#39;word&#39;, topn=5)
model.save(&#39;word2vec_model&#39;)
loaded_model = Word2Vec.load(&#39;word2vec_model&#39;)
loaded_vector = loaded_model.wv[&#39;word&#39;]
loaded_similar_words = loaded_model.wv.most_similar(&#39;word&#39;, topn=5)
print(&quot;Vector for &#39;word&#39;:\n&quot;, vector)
print(&quot;Similar words to &#39;word&#39;:\n&quot;, similar_words)
print(&quot;Vector for &#39;word&#39; from loaded model:\n&quot;, loaded_vector)
print(&quot;Similar words to &#39;word&#39; from loaded model:\n&quot;, loaded_similar_words)
```

Output:

```
(python django) PS D:\sem7\DL-NLP practical> python -u "d:\sem7\DL-NLP practical\pra
python -u "d:\sem7\DL-NLP practical\pract 10.py"
[nltk_data] Downloading package punkt_tab to
[nltk_data] C:\Users\varun\AppData\Roaming\nltk_data...
[nltk_data] Unzipping tokenizers\punkt_tab.zip.
Vector for 'word':
[-8.6187059e-03  3.6663094e-03  5.1988977e-03  5.7409322e-03
 7.4660075e-03 -6.1679049e-03  1.1055992e-03  6.0476447e-03
-2.8400451e-03 -6.1744275e-03 -4.0912686e-04 -8.3688125e-03
-5.6006135e-03  7.1033947e-03  3.3514623e-03  7.2260187e-03
 6.8011461e-03  7.5302608e-03 -3.7892631e-03 -5.6426885e-04
 2.3482488e-03 -4.5191990e-03  8.3901351e-03 -9.8580532e-03
 6.7625167e-03  2.9152690e-03 -4.9325339e-03  4.3986044e-03
-1.7403256e-03  6.7124763e-03  9.9633327e-03 -4.3619038e-03
-5.9845758e-04 -5.6969114e-03  3.8513471e-03  2.7860443e-03
 6.8910471e-03  6.1006858e-03  9.5366742e-03  9.2745312e-03
 7.8989482e-03 -6.9900220e-03 -9.1571957e-03 -3.5453803e-04
-3.0987952e-03  7.8932224e-03  5.9383572e-03 -1.5461525e-03
 1.5108582e-03  1.7904930e-03  7.8173876e-03 -9.5094036e-03
-2.0441553e-04  3.4684818e-03 -9.3982613e-04  8.3806682e-03
 9.0097468e-03  6.5359757e-03 -7.1218295e-04  7.7109039e-03
-8.5343206e-03  3.2054393e-03 -4.6385936e-03 -5.0884574e-03
 3.5894646e-03  5.3709918e-03  7.7688308e-03 -5.7665538e-03
 7.4333218e-03  6.6250931e-03 -3.7101361e-03 -8.7443022e-03
 5.4369238e-03  6.5101241e-03 -7.8766455e-04 -6.7084329e-03
-7.0842202e-03 -2.4971897e-03  5.1445863e-03 -3.6654309e-03
-9.3705216e-03  3.8273896e-03  4.8845517e-03 -6.4288019e-03
 1.2069566e-03 -2.0752454e-03  2.5317728e-05 -9.8824045e-03
 2.6923865e-03 -4.7500134e-03  1.0887211e-03 -1.5764812e-03
 2.1961369e-03 -7.8811655e-03 -2.7176354e-03  2.6616962e-03
 5.3471457e-03 -2.3918226e-03 -9.5104175e-03  4.5059794e-03]
Similar words to 'word':
[['hello', 0.1885661631822586], ('popular', 0.16076910495758057), ('language', 0.15917937457561493), ('library', 0.13726547360420227), ('are', 0.12295848876237869)]
Vector for 'word' from loaded model:
[-8.6187059e-03  3.6663094e-03  5.1988977e-03  5.7409322e-03
 7.4660075e-03 -6.1679049e-03  1.1055992e-03  6.0476447e-03
-2.8400451e-03 -6.1744275e-03 -4.0912686e-04 -8.3688125e-03
-5.6006135e-03  7.1033947e-03  3.3514623e-03  7.2260187e-03
 6.8011461e-03  7.5302608e-03 -3.7892631e-03 -5.6426885e-04
 2.3482488e-03 -4.5191990e-03  8.3901351e-03 -9.8580532e-03
 6.7625167e-03  2.9152690e-03 -4.9325339e-03  4.3986044e-03]
```

[illegible]